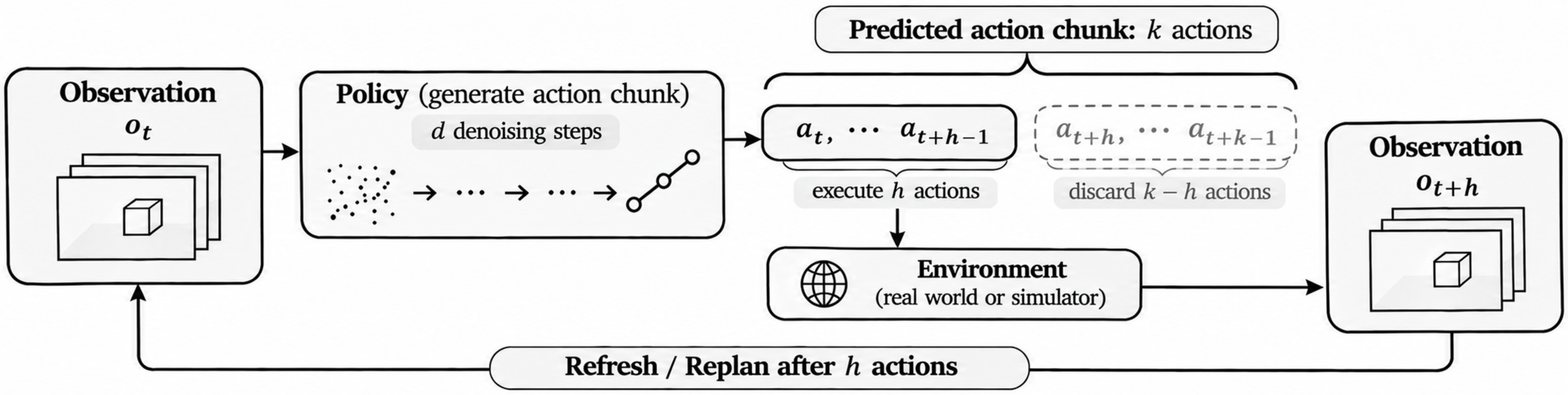
Refinement vs. Refresh

Compute Allocation for Closed-Loop Robot Policies

Alba María Téllez Fernández, MAAT Institute (alba.tellez@maatinstitute.com)

Chunked Closed-Loop Policy



Parameters | k : number of predicted actions per chunk (chunk length) | h : executed actions per replan ($h \leq k$) | d : denoising steps (refinement)

Synchronous closed-loop execution. Each policy call predicts k actions; only the first h are executed before obtaining a new observation and replanning.

Setup (held-out)

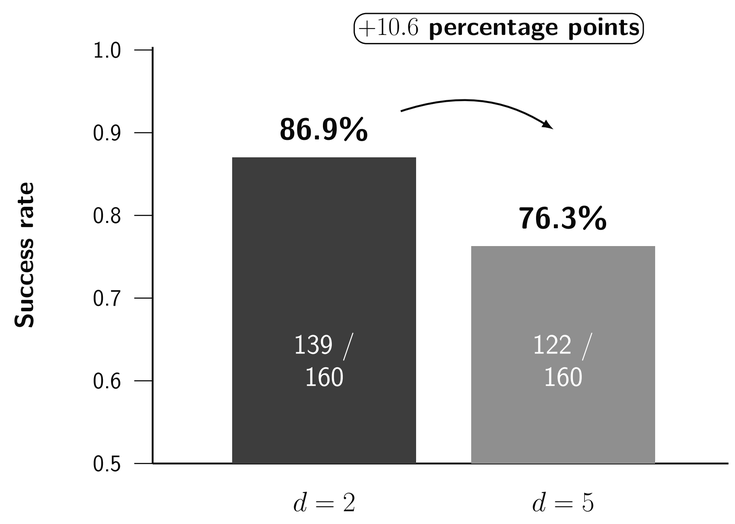
SmolVLA (frozen) $K = 50,\ d \in \{2, 5\}$

LIBERO (simulation) $h \in \{2, 5\}$ 8 tasks, 1,120 attempts

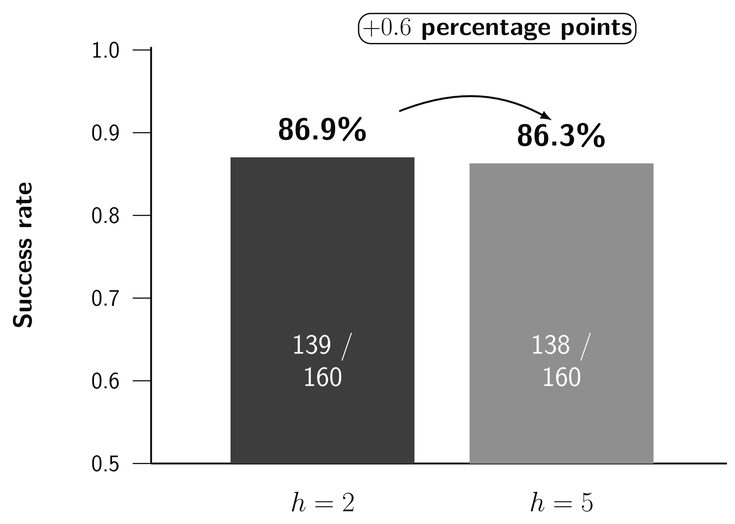
?

Under a fixed inference budget, should we allocate compute to additional denoising steps (**refinement**) or to more frequent observation-conditioned replanning (**refresh**)?

Result 1: Fewer Denoising Steps Improve Success



Result 2: Less Frequent Refresh Is Much Cheaper



What the results mean

Takeaway

RoboPAD is a worked example of how to study closed-loop inference allocation: define refinement and refresh controls, measure complete inference cost, freeze comparisons, and evaluate success and expenditure together.

Interpretation and limits

SmolVLA provides the case study, not a universal allocation rule. The framework transfers to other policies, but each policy requires within-model confirmation; arithmetic is not deployment latency, and equivalent quality is not established.